Przedmiot: Przegląd metod i narzędzi AI Cel: Analiza zbioru danych Iris przy użyciu algorytmu KNN	Prowadzący: mgr. Inż. Łukasz Szarejko
Wykonawca: Martyna Misarska nr.119122 grupa4, Filip Piotrowski nr.119072 grupa4	Data: 26.03.2026r.

Informacje o zbiorze danych

Do naszego zadania skorzystaliśmy z zbioru danych Iris dotyczącego kosaćców. Zestaw dzieli się na 3 klasy, które odnoszą się do różnych rodzajów irysów. Składa się on z 150 próbek podzielonych równo na te 3 klasy, a w każdej z klas znajduje się po 50 próbek:

- Iris setosa
- Iris versicolor
- Iris virginica

Dlatego, że klasy są doskonalone zrównoważone, ten zbiór danych stanowi idealne źródło do uczenia się oraz weryfikacji algorytmów klasyfikacyjnych. Każda próbka charakteryzuje się 4 cechami numerycznymi wyrażonymi w centymetrach:

1. Długość działki kielicha (sepal length)
2. Szerokość działki kielicha (sepal width)
3. Długość płatka (petal length)
4. Szerokość płatka (petal width)

Analiza danych oraz dostępnej literatury wskazuje, że klasa Setosa może być łatwo oddzielona od pozostałych ze względu na znacznie mniejsze rozmiary kwiatów. Gatunki Versicolor i Virginica są do siebie bardziej zbliżone, a ich cechy nieco się pokrywają.

Algorytm KNN

Czym jest algorytm KNN?

K-Najbliższych sąsiadów to jeden z najprostszych, ale bardzo efektywnych algorytmów w obszarze nadzorowanego uczenia maszynowego, stosowany głownie w problemach klasyfikacyjnych. W trakcie etapu uczenia algorytm nie tworzy żadnego modelu matematycznego, lecz jedynie przechowuje całość danych uczących w pamięci . Dopiero gdy napotyka nową, nieznaną próbkę, algorytm przeprowadza działania w czasie rzeczywistym, aby przyporządkować ją do właściwej grupy.

Zaimplementujmy do klasyfikacji przedstawiony powyżej, algorytm K-Najbliższych sąsiadów (K-Nearest Neighbors).

Zasady działania własnej implementacji :

- **Distance Metric (miara odległości)** - wykorzystamy tradycyjna odległość euklidesową do oceny dystansu pomiędzy punkami w czterowymiarowej przestrzeni cech
- **Neighbor Selection(wybór sąsiadów)** - dla każdej analizowanej próbki testowej, algorytm ustala odległość do wszystkich punktów w zbirze danych uczących oraz wybiera k punktów o

najmniejszym dystansie. W tym eksperymencie przetestowaliśmy algorytm dla 6 różnych wartości parametru $k \in \{1,3,5,9,19,30\}$, aby zbadać jego skuteczność klasyfikacji.

- **Majority Voting(głosowanie)** - algorytm zlicza które klasy sąsiadów znalazły się w wybranym k i przypisuje próbę tę klasę, która występuje najczęściej.
- Dane przed klasyfikacją zostały losowo przemieszane i podzielone na zbiór treningowy 60% oraz testowy 40%
- **Powtarzalność wyników -** Ze względu na to, że proces nauki i testowania modelu wymaga losowego mieszania oraz podziału danych (funkcja $train_test_split$), w kodzie wykorzystaliśmy ziarno generowania liczb pseudolosowych z pakietu NumPy: $np.random.seed(30)$. Ten krok ma na celu zamrożenie procesu losowania, co zapewnia całkowitą powtarzalność eksperymentu. Dzięki temu, przy każdym uruchomieniu programu, podział na zestaw treningowy i testowy pozostaje taki sam, co eliminuje wahania ostatecznych wyników i umożliwia rzetelna oraz spójną analizę efektywności algorytmu.

Metryki oceny modelu

Aby dokładnie ocenić działanie stworzonej funkcji KNN oraz zbadać wpływ parametru k na jakość predykacji, stworzymy od podstaw obliczenia metryk opartych na macierzy pomyłek. Analiza wyników według klas pozwala na rozumienie, z jakim typem kosaćca algorytm najlepiej sobie radzi, a gdzie występują błędy. Wykorzystane metryki:

Accuracy, czyli dokładność to podstawowa miara, która ocenia jaki procent został poprawnie sklasyfikowany w całym zbiorze testowym. W przypadku Iris, gdzie każda klasa ma po 50 próbek, ta metryka jest wiarygodna

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN}$$

Precision, czyli precyzja to metryka, która odpowiada na pytanie „Ile w wszystkich kwiatów, które algorytm zaklasyfikował jako klasę X, rzeczywiście do niej należy?”. Jeżeli precyzja jest wysoka to model rzadko generuje fałszywie pozytywne.

$$Precision = \frac{TP}{TP + FP}$$

Recall, czyli czułość to miara, która odpowiada na pytanie „Jaki procent rzeczywistych kwiatów z klasy X algorytm zdołał poprawnie zidentyfikować? Wysoka czułość sugeruje, że model dobrze rozpoznaje próbki danej klasy.

$$Recall = \frac{TP}{TP + FN}$$

F1-score, czyli średnia harmoniczna precyzji i czułości. Używa się jej, aby uzyskać, jedną, skondensowaną ocenę wydajności modelu. Średnia harmoniczna mocno karze skrajne wartości, co oznacza, że F1-score będzie wysoka tylko wtedy, gdy zarówno precyzja jak i czułość są na wysokim poziomie.

$$F1 = 2 \cdot \frac{Precision \cdot Recall}{Precision + Recall}$$

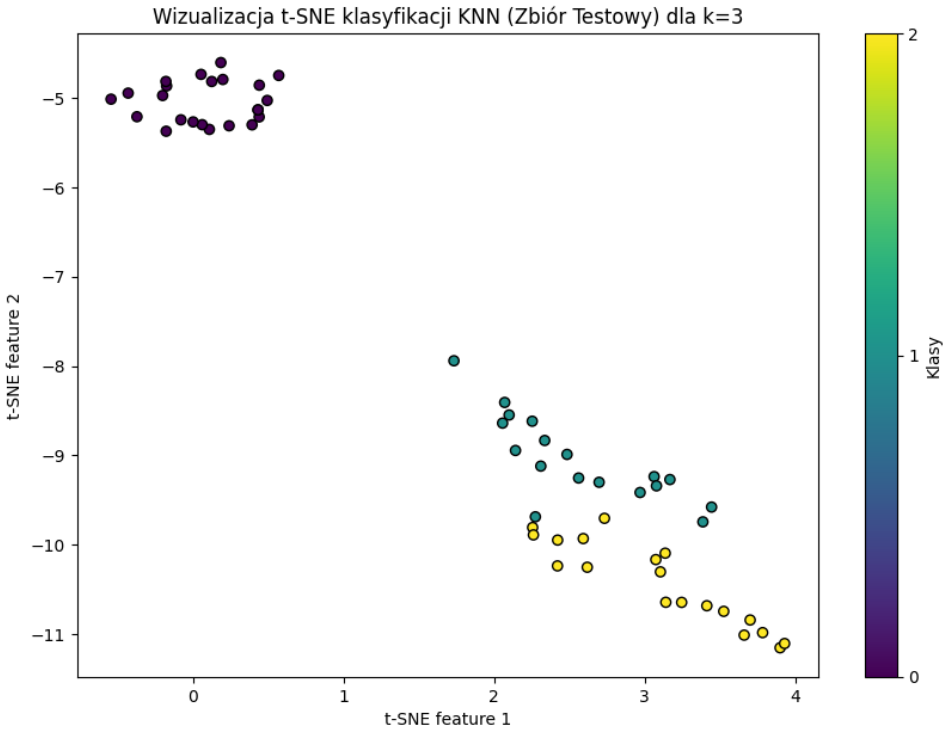
Gdzie: TP- True Positives, TN-True Negatives, FP -Falese Positives, FN- False Negatives

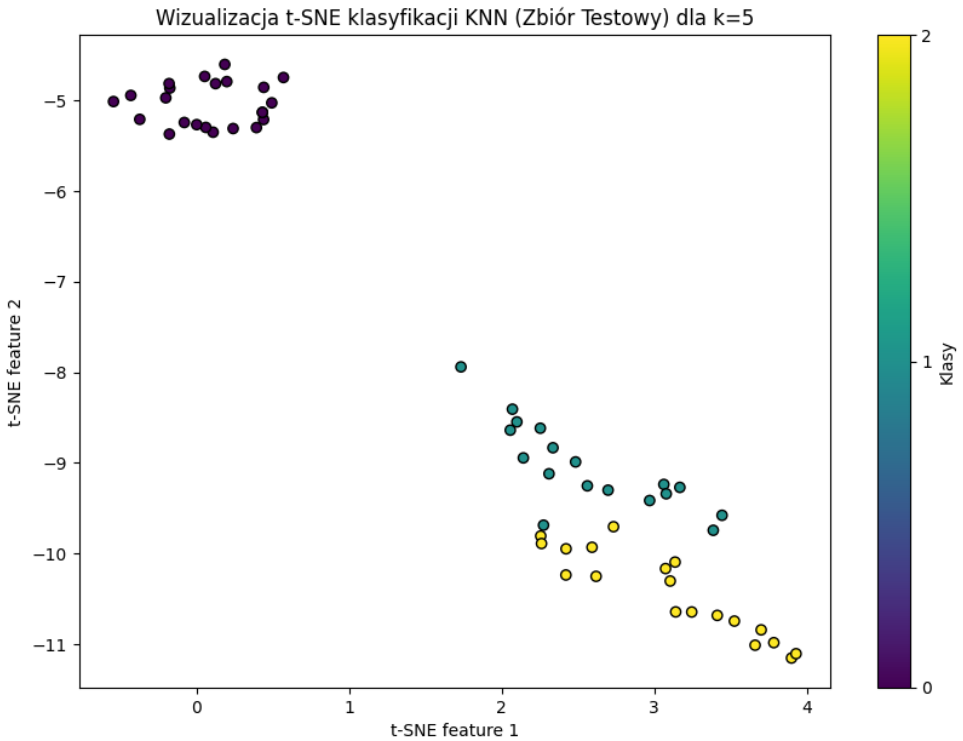
Szczegółowe wyniki dla poszczególnych wartości parametru k:

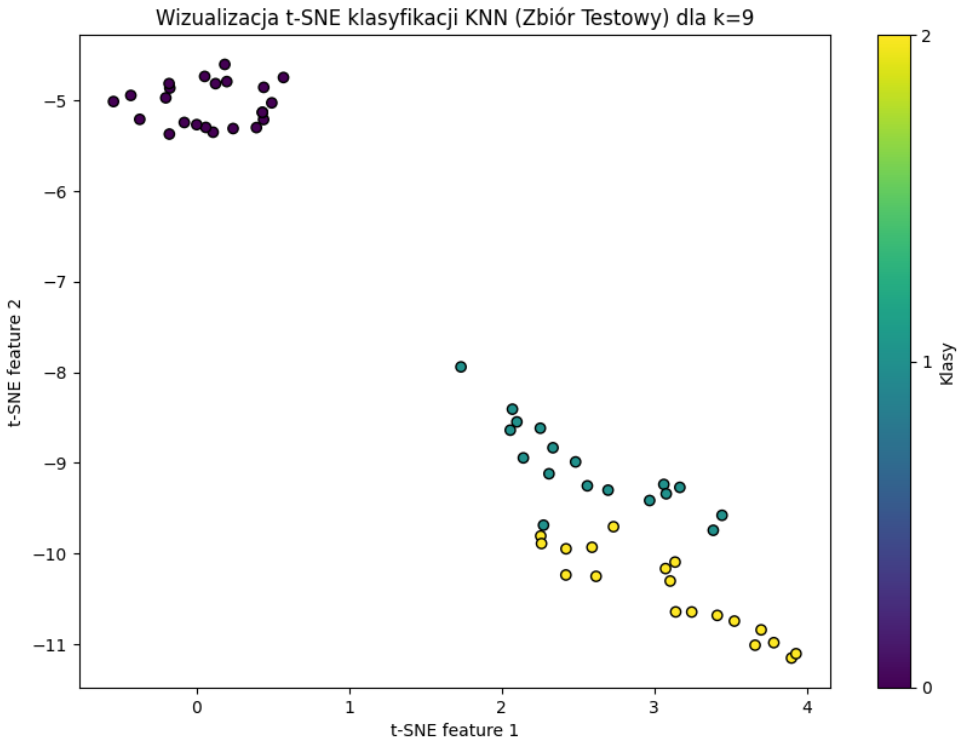
	Precision	Recall	F1-score		Precision	Recall	F1-score
k=1	Ogólne Accuracy: 98.33%			k=9	Ogólne Accuracy:98.33%		
klasa 1 (Setosa)	1.00	1.00	1.00	klasa 1 (Setosa)	1.00	1.00	1.00
klasa 2 (Versicolor)	0.94	1.00	0.97	klasa 2 (Versicolor)	0.94	1.00	0.97
klasa 3(Virginica)	1.00	0.95	0.97	klasa 3(Virginica)	1.00	0.95	0.97
k=3	Ogólne Accuracy: 98.33%			k=19	Ogólne Accuracy: 93.33%		
klasa 1 (Setosa)	1.00	1.00	1.00	klasa 1 (Setosa)	1.00	1.00	1.00
klasa 2 (Versicolor)	0.94	1.00	0.97	klasa 2 (Versicolor)	0.84	0.94	0.89
klasa 3(Virginica)	1.00	0.95	0.97	klasa 3(Virginica)	0.94	0.85	0.89
k=5	Ogólne Accuracy: 98.33%			k=30	Ogólne Accuracy: 90.00%		
klasa 1 (Setosa)	1.00	1.00	1.00	klasa 1 (Setosa)	1.00	1.00	1.00
klasa 2 (Versicolor)	0.94	1.00	0.97	klasa 2 (Versicolor)	0.76	0.94	0.84
klasa 3(Virginica)	1.00	0.95	0.97	klasa 3(Virginica)	0.94	0.75	0.83

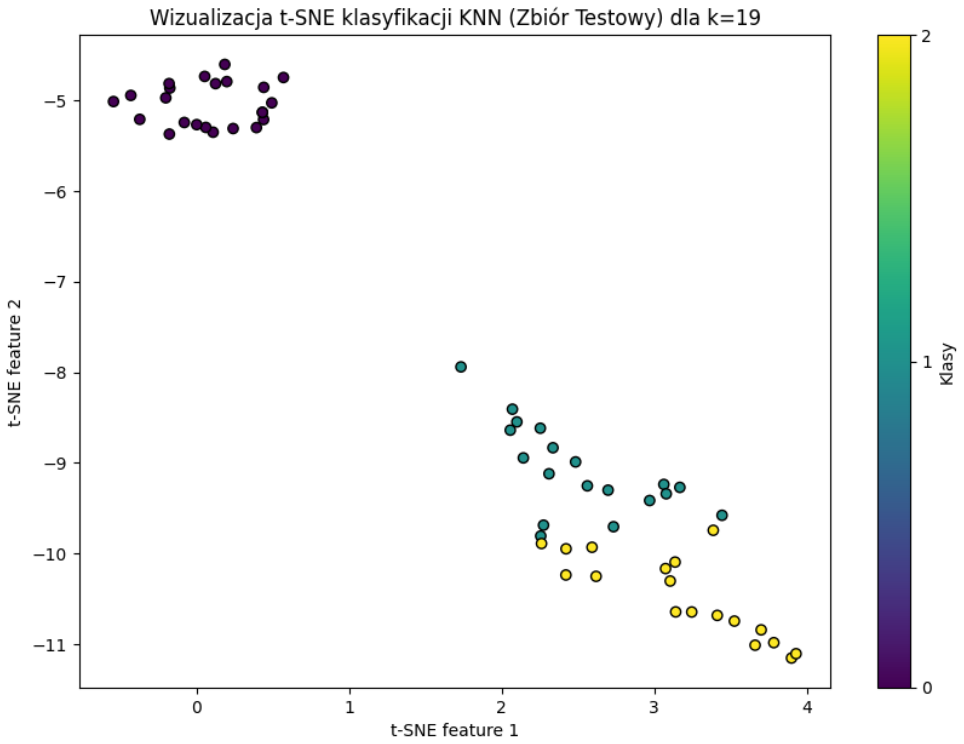
Wizualizacja przy wykorzystaniu algorytmu t-SNE

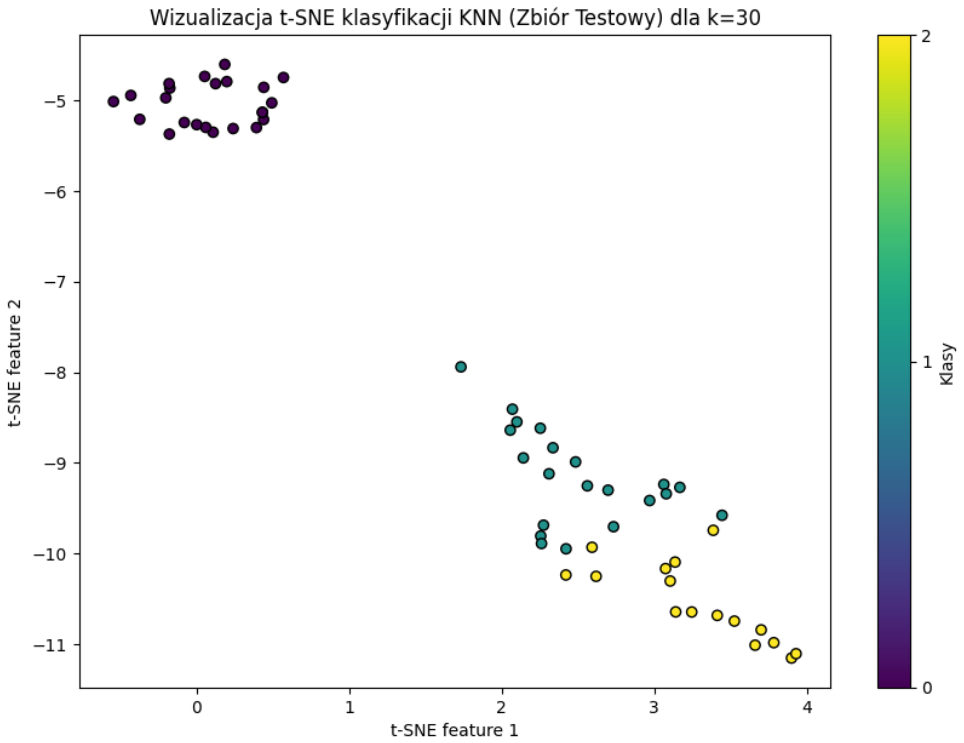
Pierwotne dane zawierają cztery wymiary(cechy), z tego powodu, do ich przedstawienia graficznego zastosujemy algorytm t-SNE. Ten algorytm zmniejszy liczbę wymiarów do formatu 2D, przy zachowaniu lokalnych relacji oraz podobieństw pomiędzy punkami. Ponieważ parametry rzutowania (w tym ziarno losowości) uwzględniliśmy, że były stałe, współrzędne punktów na każdym z poniższych wykresów są identyczne. Zmieniają się tylko ich kolory, które opowiadają klasom przewidzianym przez model dla danej wartości k. Pozwala to nam na bezpośrednią, wizualną obserwacje, jak wielkość sąsiedztwa wpływa na granice decyzyjne modelu.


Wizualizacja t-SNE klasyfikacji KNN (Zbiór Testowy) dla k=3
t-SNE feature 2
t-SNE feature 1
Klasy
−5
−6
−7
−8
−9
−10
−11
0
1
2
3
4
0
1
2


Wizualizacja t-SNE klasyfikacji KNN (Zbiór Testowy) dla k=5
t-SNE feature 2
t-SNE feature 1
Klasy
−5
−6
−7
−8
−9
−10
−11
0
1
2
3
4
0
1
2


Wizualizacja t-SNE klasyfikacji KNN (Zbiór Testowy) dla k=9
t-SNE feature 2
t-SNE feature 1
Klasy
−5
−6
−7
−8
−9
−10
−11
0
1
2
3
4
0
1
2


Wizualizacja t-SNE klasyfikacji KNN (Zbiór Testowy) dla k=19
t-SNE feature 2
t-SNE feature 1
Klasy
−5
−6
−7
−8
−9
−10
−11
0
1
2
3
4
0
1
2



Wspólne obserwacje dla wykresów:

Wszystkie wykresy ukazują nam dwa ważne skupienia. Pierwsze z nich w lewym górnym rogu **(kolor fioletowy)** odnosi się do klasy **Setosa**. Skupisko to jest wyraźnie oddzielone od pozostałych, co prowadzi do idealnych metryk: Precision=1.00, Recall=1.00, w każdym z eksperymentów. Drugie skupisko w prawym dolnym rogu składa się z wymieszanych próbek klas Versicolor (kolor morski) i **Virginica (kolor żółty).** To właśnie na granicy pomiędzy tymi klasami występują zmiany w prognozach.

Zauważamy **klasę stabilności** dla $k \in \{3,5,9\}$. W tym zakresie model osiąga maksimum oraz stabilność skuteczności. W zestawieniu metryk możemy zauważyć całkowitą dokładność dla każdej z tych wartości na poziomie 98.33%. Metryki dla poszczególnych klas nie zmieniają się znacząco, wskazując jedynie niewielkie problemy na granicy pomiędzy klasami Versicolor i Virginica.

Granica decyzyjna w prawym dolnym rogu jest bardzo precyzyjna. Algorytm skutecznie rozpoznaje lokalne skupiska punktów o kolorze morskim i żółtym, pozostając nieufnym dla pojedynczych sąsiadów.

Faza nadmiernego uogólnienia dla $k = 19\ i\ k = 30$. Kiedy wartość k staje się zbyt wysoka w porównaniu do wielkości zbioru treningowego, model zaczyna uwzględniać zbyt szerokie sąsiedztwo, co prowadzi do rozmycia granic pomiędzy klasami. To zjawisko możemy zauważyć na wykresach i wynikach tabeli o dużym k. Dla $k = 19$ Skuteczność spada do 93.33%, zaś przy $k = 30$ znacząco spada do 90.00%.

Wnioski dla wykresów:

Porównując prawy dolny róg wykresu dla $k = 9$ z tym dla $k = 30$, widać wyraźnie, że wiele punktów wcześniej oznaczonych jako żółte teraz uzyskało kolor morski. Dzieje się tak, ponieważ przy $k = 30$ algorytm zasięga rady aż 30 sąsiadów naraz. Gdy klasa 2 jest w danym obszarze trochę bardziej skoncentrowana, zaczyna dominować nad punktami klasy 3, całkowicie ignorując lokalne szczegóły. Podsumowując wizualizację: Wybór odpowiedniego k (w tym przypadku w zakresie 3-9) ma kluczowe znaczenie. **Zbyt duża wartość k** prowadzi do **rozmywania** granic oraz faworyzowania klas z przewagą liczebną w szerszym kontekście analizy.

Wnioski i podsumowanie

Wykonanie tego projektu przybliżyła nam działanie algorytmu KNN. Własna jego wersja skutecznie klasyfikuje dane wielkoklasowe na zbiorze Iris, niezależnie od wybranej wartości k. Klasa 1 Setosa jest zawsze poprawnie klasyfikowana w każdym z przeprowadzonych testów. Dzieję się tak ponieważ jej próbki w przestrzeni cech są wyraźnie oddzielone od innych gatunków. Błędy w klasyfikacji dotyczą jedynie gatunków Versicolor i Virginica, których klastery są blisko siebie lub się ze sobą przenikają, co można zobaczyć także na wykresach.

Porównując wyniki, możemy dostrzec, że najwyższa skuteczność (Accuracy) została osiągnięta przy wartości $k \in \{3,5,9\}$. Zbyt mała wartość k np. k=1 sprawia, że model jest narażony na zakłócenia. Natomiast zbyt duża wartość k może spowodować, że granice decyzyjne staną się zbyt elastyczne, co może prowadzić do dominacji klasy o wyższym zagęszczeniu na granicach klas. Wprowadzenie ziarna losowości np.random.seed(30) pozwoliło nam na wyeliminowanie wpływu przypadkowego podziału danych, co zapewniło, że zauważone różnice w metrykach wynikają jedynie ze zmiany parametru k.